

Gestión de bibliotecas

-

Documentación

Índice:

1- Introducción	3
2- Conexión con la base de datos	3
3- Desarrollo de consultas sobre la base de datos	6
4- Gestión de colecciones	22
5- Gestión de documentos XML	26
6- Conclusión	29

1- Introducción

La gestión de bibliotecas es un proceso esencial para la organización y administración eficiente de colecciones de libros, préstamos y usuarios. Este documento proporciona una visión detallada sobre la implementación de un sistema de gestión de bibliotecas basado en bases de datos XML utilizando BaseX.

En las siguientes secciones, se describen los componentes clave del sistema, incluyendo la conexión con la base de datos, el desarrollo de consultas, la gestión de colecciones y la administración de documentos XML. Se presentan también estructuras de datos y clases diseñadas para garantizar la eficiencia en la manipulación de la información.

Este documento servirá como una guía técnica para comprender la arquitectura del sistema, facilitando su mantenimiento y posibles mejoras futuras.

2- Conexión con la base de datos

Para realizar la conexión con la base de datos se ha creado una clase exclusiva BaseXSessionUtil, que se encarga de realizar la conexión y de devolvernos los objetos de tipo ClientServer para poder operar con la base de datos.

```
package com.spachecor.ejerciciofinalsgb.model.repository;

import
com.spachecor.ejerciciofinalsgb.model.documents.DocumentosManager;
import com.spachecor.ejerciciofinalsgb.model.entity.Libro;
import com.spachecor.ejerciciofinalsgb.model.entity.Prestamo;
import com.spachecor.ejerciciofinalsgb.model.entity.Usuario;
import com.thoughtworks.xstream.core.BaseException;
import org.basex.api.client.ClientSession;

import java.awt.print.Book;
import java.io.IOException;
import java.nio.file.Paths;

/**
 * Clase BaseXSessionUtil que define como crear la sesion para
 * conectarse a la base de datos y operar con ella.
 * @author Selene
 * @version 1.0

```

```

*/
public class BaseXSessionUtil {
    private static final String HOST = "localhost";
    private static final int PORT = 1984;
    private static final String USER = "admin";
    private static final String PASSWORD = "admin";
    public static final String DATABASE =
"librarywithcollections";

    /**
     * Funcion que crea la sesion para acceder a la base de
    datos XML nativa con BaseX y abre la base de datos concreta.
     * @return El objeto ClientSession necesario y configurado
    para hacer consultas a la base de datos
     */
    public static ClientSession getSession() {
        try{
            ClientSession clientSession = new
ClientSession(BaseXSessionUtil.HOST, BaseXSessionUtil.PORT,
BaseXSessionUtil.USER, BaseXSessionUtil.PASSWORD);
            //si la bbdd no existe, la creamos y le metemos
            algunas entidades de ejemplo

            if(!Boolean.parseBoolean(clientSession.query("db:exists('"+Bas
eXSessionUtil.DATABASE+"')").execute())){
                clientSession.execute("create db
"+"BaseXSessionUtil.DATABASE);
                BaseXSessionUtil.crearDatosDePrueba();
            }

            clientSession.execute("open
"+"BaseXSessionUtil.DATABASE);
            return clientSession;
        } catch (IOException e) {
            System.err.println("Error: "+e.getMessage());
            System.out.println("Finalizando programa...");
            System.exit(1);
            return null;
        }
    }

    /**
     * Funcion que crea un backup de de base de datos actual
     * @param session El objeto ClientSession para ejecutar la
    creacion del backup

```

```

        * @throws IOException Excepcion que puede lanzar en el
proceso
    */
    public static void iniciarBackup(ClientSession session)
throws IOException {
        session.execute("create backup
"+BaseXSessionUtil.DATABASE);
    }

    /**
     * Funcion que restaura la base de datos al backup anterior
     * @param session El objeto ClientSession para ejecutar la
restauracion al ultimo backup
     * @throws IOException Excepcion que puede lanzar
     */
    public static void restaurarBackup(ClientSession session)
throws IOException {
        session.execute("restore "+BaseXSessionUtil.DATABASE);
    }

    /**
     * Funcion que genera una serie de entidades de prueba
     */
    private static void crearDatosDePrueba() {
        DocumentosManager.agregarDocumento(Libro.class,
Paths.get("src/main/resources/com/spachecor/ejerciciofinalsgb/
dbexample/books/book1.xml").toAbsolutePath().toString(),
"fiction");
        DocumentosManager.agregarDocumento(Libro.class,
Paths.get("src/main/resources/com/spachecor/ejerciciofinalsgb/
dbexample/books/book2.xml").toAbsolutePath().toString(),
"fiction");
        DocumentosManager.agregarDocumento(Libro.class,
Paths.get("src/main/resources/com/spachecor/ejerciciofinalsgb/
dbexample/books/book3.xml").toAbsolutePath().toString(),
"nonfiction");
        DocumentosManager.agregarDocumento(Libro.class,
Paths.get("src/main/resources/com/spachecor/ejerciciofinalsgb/
dbexample/books/book4.xml").toAbsolutePath().toString(),
"nonfiction");
        DocumentosManager.agregarDocumento(Libro.class,
Paths.get("src/main/resources/com/spachecor/ejerciciofinalsgb/
dbexample/books/book5.xml").toAbsolutePath().toString(),
"children");
    }

```

```

        DocumentosManager.agregarDocumento(Usuario.class,
Paths.get("src/main/resources/com/spachecor/ejerciciofinalsgb/
dbexample/users/user1.xml").toAbsolutePath().toString(),
null);

        DocumentosManager.agregarDocumento(Usuario.class,
Paths.get("src/main/resources/com/spachecor/ejerciciofinalsgb/
dbexample/users/user2.xml").toAbsolutePath().toString(),
null);

        DocumentosManager.agregarDocumento(Usuario.class,
Paths.get("src/main/resources/com/spachecor/ejerciciofinalsgb/
dbexample/users/user3.xml").toAbsolutePath().toString(),
null);

        DocumentosManager.agregarDocumento(Prestamo.class,
Paths.get("src/main/resources/com/spachecor/ejerciciofinalsgb/
dbexample/loans/loan1.xml").toAbsolutePath().toString(),
null);

        DocumentosManager.agregarDocumento(Prestamo.class,
Paths.get("src/main/resources/com/spachecor/ejerciciofinalsgb/
dbexample/loans/loan2.xml").toAbsolutePath().toString(),
null);

        DocumentosManager.agregarDocumento(Prestamo.class,
Paths.get("src/main/resources/com/spachecor/ejerciciofinalsgb/
dbexample/loans/loan3.xml").toAbsolutePath().toString(),
null);
    }
}

```

3- Desarrollo de consultas sobre la base de datos

Para las consultas sobre la base de datos, se ha asumido que la estructura de la base de datos está estructurada en documentos y “colecciones”. Las colecciones como tal no existen en BaseX, pero se han simulado con rutas internas.

Con todo esto, se ha creado un CRUD genérico, para que cualquier entidad que herede de la clase Entidad pueda usarlo, siempre y cuando implemente un repositorio donde configura la estructura(paso que ha sido inevitable).

Para esto tenemos los modelos, que todos heredan de la clase Entidad, que es una clase abstracta genérica.

```

package com.spachecor.ejerciciofinalsgb.model.entity;

/**

```

```

* Clase Entidad, que es una clase abstracta genérica que
define como deben ser las entidades
* @param <T> El tipo de entidad
* @author Selene
* @version 1.0
*/
public abstract class Entidad<T extends Entidad> implements
Comparable<T>{
    /**
     * Funcion para obtener el id de la entidad
     * @return El id de la entidad
     */
    public abstract Integer getId();
}

```

Y sus herederas (a partir de ahora, todos los ejemplos se mostrarán con Libro, aunque contamos con Libro, Usuario y Préstamo).

```

package com.spachecor.ejerciciofinalsgb.model.entity;

/**
 * Clase Libro que define las propiedades y el comportamiento
del objeto Libro
 * @author Selene
 * @version 1.0
 */
public class Libro extends Entidad<Libro> {
    private Integer id;
    private String titulo;
    private String autor;
    private Integer anioPublicacion;
    private String categoria;
    private String subCategoria;
    private Long isbn;
    private String editorial;
    private Integer numeroPaginas;
    private Integer numeroCopiasDisponibles;

    public Libro() {}

    public Libro(Integer id, String titulo, String autor,
Integer anioPublicacion, String categoria, String
subCategoria, Long isbn, String editorial, Integer
numeroPaginas, Integer numeroCopiasDisponibles) {

```

```
        this.id = id;
        this.titulo = titulo;
        this.autor = autor;
        this.anioPublicacion = anioPublicacion;
        this.categoria = categoria;
        this.subCategoria = subCategoria;
        this.isbn = isbn;
        this.editorial = editorial;
        this.numeroPaginas = numeroPaginas;
        this.numeroCopiasDisponibles = numeroCopiasDisponibles;
    }

    public void setId(Integer id) {
        this.id = id;
    }

    public String getTitulo() {
        return titulo;
    }

    public void setTitulo(String titulo) {
        this.titulo = titulo;
    }

    public String getAutor() {
        return autor;
    }

    public void setAutor(String autor) {
        this.autor = autor;
    }

    public Integer getAnioPublicacion() {
        return anioPublicacion;
    }

    public void setAnioPublicacion(Integer anioPublicacion) {
        this.anioPublicacion = anioPublicacion;
    }

    public String getCategoria() {
        return categoria;
    }
}
```



```

public void setCategoria(String categoria) {
    this.categoria = categoria;
}

public String getSubCategoria() {
    return subCategoria;
}

public void setSubCategoria(String subCategoria) {
    this.subCategoria = subCategoria;
}

public Long getIsbn() {
    return isbn;
}

public void setIsbn(Long isbn) {
    this.isbn = isbn;
}

public String getEditorial() {
    return editorial;
}

public void setEditorial(String editorial) {
    this.editorial = editorial;
}

public Integer getNumeroPaginas() {
    return numeroPaginas;
}

public void setNumeroPaginas(Integer numeroPaginas) {
    this.numeroPaginas = numeroPaginas;
}

public Integer getNumeroCopiasDisponibles() {
    return numeroCopiasDisponibles;
}

    public void setNumeroCopiasDisponibles(Integer
numeroCopiasDisponibles) {
    this.numeroCopiasDisponibles = numeroCopiasDisponibles;
}

```

```

@Override
public String toString() {
    return this.titulo;
}

@Override
public Integer getId() {
    return id;
}

@Override
public int compareTo(Libro t) {
    if(
        this.autor.compareTo(t.getAutor())==0
        &&this.titulo.compareTo(t.getTitulo())==0
        &&this.anioPublicacion.compareTo(t.getAnioPublicacion())==0
    )return 0;
    else return -1;
}
}

```

Después de configurar los modelos para que sean lo más genéricos posibles, se ha creado un repositorio genérico que implementa una interfaz que define los métodos CRUD:

```

package com.spachecor.ejerciciofinalsgb.model.dao;

import com.spachecor.ejerciciofinalsgb.model.entity.Entidad;

import java.util.List;
import java.util.Optional;

/**
 * Interfaz generica GenericDAO que se encarga de definir las
 * funciones para el acceso a la base de datos
 * para todas las entidades
 * @param <T> T será una clase que herede de Entidad
 * @author Selene
 * @version 1.0
 */
public interface GenericDAO<T extends Entidad> {
    /**

```

```

    * Funcion que devuelve una lista de Entidades del tipo T
    * @return Lista de Entidades del tipo T
    */
    List<T> listar();

    /**
     * Funcion que busca una Entidad concreta del tipo T a
    partir de su id
     * @param id El id de la Entidad
     * @return Un objeto Optional que puede contener la Entidad
    que coincide con el id
     */
    Optional<T> buscarPorId(Integer id);

    /**
     * Funcion que crea una Entidad en la base de datos a
    partir de la Entidad pasada por parametro
     * @param url La url interna del programa donde se ubica el
    fichero que contiene la Entidad
     * @param subColeccion El nombre de la coleccion donde lo
    almacenaremos
     */
    void crear(String url, String subColeccion);

    /**
     * Funcion que actualiza una Entidad en la base de datos
     * @param t La Entidad del tipo T a actualizar
     */
    void actualizar(T t);

    /**
     * Funcion que elimina una Entidad de la base de datos
     * @param t La Entidad a eliminar del tipo T
     */
    void eliminar(T t);
}

```

```

package com.spachecor.ejerciciofinalsgb.model.dao;

import com.spachecor.ejerciciofinalsgb.model.entity.Entidad;
import com.spachecor.ejerciciofinalsgb.model.mapper.Mapper;

```

```

import
com.spachecor.ejerciciofinalsgb.model.repository.BaseXSessionU
til;
import org.basex.api.client.ClientSession;
import org.w3c.dom.Document;
import org.w3c.dom.Element;
import org.w3c.dom.NodeList;

import javax.xml.parsers.DocumentBuilder;
import javax.xml.parsers.DocumentBuilderFactory;
import javax.xml.transform.OutputKeys;
import javax.xml.transform.Transformer;
import javax.xml.transform.TransformerFactory;
import javax.xml.transform.dom.DOMSource;
import javax.xml.transform.stream.StreamResult;
import java.io.ByteArrayInputStream;
import java.io.StringWriter;
import java.nio.charset.StandardCharsets;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

/**
 * Clase abstracta EntidadGenericDAOImpl que tiene de utilidad
 el ser el DAO genérico de todas las Entidades que
 * hereden de Entidad. Para usarse hay que crear una clase
 hija, que herede esta clase, y así asignar el Mapper,
 * el CollectionPath y el EntityTag.
 * @param <T> El tipo de Entidad que consume la
 EntidadGenericDAOImpl
 * @author Selene
 * @version 1.0
 */
public abstract class EntidadGenericDAOImpl<T extends Entidad>
implements GenericDAO<T> {
    /**
     * Funcion que hace que cada implementacion de esta clase
 proporcione su Mapper correspondiente
     * @return El Mapper correspondiente a la Entidad de tipo T
     */
    protected abstract Mapper<T> getMapper();

    /**

```

```

    * Funcion que devuelve la ruta raiz en la base de datos
XML donde se almacenan los nodos de la Entidad tipo T
    * @return La ruta raiz de la Entidad tipo T en la base de
datos XML
    */
    protected abstract String getCollectionPath();

    /**
    * Funcion que devuelve el nombre de la etiqueta XML que
identifica la Entidad tipo T en la base de datos XML
    * @return El nombre de la etiqueta de la Entidad tipo T
    */
    protected abstract String getEntityTag();

    @Override
    public List<T> listar() {
        String xquery =
"collection('"+BaseXSessionUtil.DATABASE+"')//"+this.getEntity
Tag()+"[starts-with(db:path(.),
'"+this.getCollectionPath()+"')]"
        return this.getListaAPartirDeQuery(xquery);
    }

    @Override
    public Optional<T> buscarPorId(Integer id) {
        try(ClientSession session =
BaseXSessionUtil.getSession()) {
            String xquery =
"collection('"+BaseXSessionUtil.DATABASE+"')//"+this.getEntity
Tag()+"[starts-with(db:path(.),
'"+this.getCollectionPath()+"')]"+"[id="+id+"]";
            //String xquery = "for $t in
"+this.getCollectionPath()+"/"+this.getEntityTag()+"[id="+id+"
] return $t";
            String resultado = session.query(xquery).execute();
            //si ha habido resultado:
            if (resultado != null &&
!resultado.trim().isEmpty()) {
                //asumimos que el xml que viene esta bien
formado porque viene solo un resultado
                return
Optional.of(getMapper().deXML(resultado));
            }
        } catch (Exception e) {

```

```

        e.printStackTrace();
    }
    //si no tenemos resultado, devolvemos un Optional vacío
    return Optional.empty();
}

@Override
public void crear(String url, String subColeccion) {
    try (ClientSession session =
BaseXSessionUtil.getSession()) {
        //creamos el backup antes que nada para realizar
una "gestion de transacciones"
        BaseXSessionUtil.iniciarBackup(session);
        //mantenemos la agregación del documento en esta
clase en vez de llamar a DocumentosManger para poder
gestionarla y hacer backup en caso necesario
        String insertQuery = "add to
"+this.getCollectionPath()+
((subColeccion!=null&&!subColeccion.isEmpty())?"/"+subColeccio
n:"") +"/ "+url;
        try{
            session.execute(insertQuery);
        }catch (Exception e){
            //si sale mal, restauramos al backup y
relanzamos la excepcion
            BaseXSessionUtil.restaurarBackup(session);
            throw e;
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
}

@Override
public void actualizar(T t) {
    try (ClientSession session =
BaseXSessionUtil.getSession()) {
        //creamos el backup antes que nada para realizar
una "gestion de transacciones"
        BaseXSessionUtil.iniciarBackup(session);
        String xml = getMapper().aXML(t);
        String updateQuery = "replace node
collection('"+BaseXSessionUtil.DATABASE+"')//"+this.getEntityT

```

```

ag()+"[starts-with(db:path(.),
'"+this.getCollectionPath()+"')][id="+t.getId()+"] with "+xml;
    try{
        session.query(updateQuery).execute();
    }catch (Exception e){
        //si sale mal, restauramos al backup y
        relanzamos la excepcion
        BaseXSessionUtil.restaurarBackup(session);
        throw e;
    }
} catch (Exception e) {
    e.printStackTrace();
}

}

@Override
public void eliminar(T t) {
    try (ClientSession session =
BaseXSessionUtil.getSession()) {
        //creamos el backup antes que nada para realizar
        una "gestion de transacciones"
        BaseXSessionUtil.iniciarBackup(session);
        String deleteQuery = "delete node
collection('"+BaseXSessionUtil.DATABASE+"')//"+this.getEntityT
ag()+"[starts-with(db:path(.),
'"+this.getCollectionPath()+"')][id="+t.getId()+"]";
        //eliminamos tambien el recurso, eliminanto to el
        rastro de nuestro registro
        String ruta = this.obtenerRutaEntidad(t);
        String deleteResource = "delete "+ruta;
        try{
            session.query(deleteQuery).execute();
            session.execute(deleteResource);
        }catch (Exception e){
            //si sale mal, restauramos al backup y
            relanzamos la excepcion
            BaseXSessionUtil.restaurarBackup(session);
            throw e;
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
}
}

```

```

/**
 * Funcion que nos genera una lista de entidades a partir
de una query que le entra por parámetro, permitiendo
 * que con cualquier consulta consigamos una lista de
entidades, según el filtro que apliquemos
 * @param query La consulta para obtener la lista de
entidades
 * @return La lista de entidades
 */
protected List<T> getListaAPartirDeQuery(String query) {
    List<T> lista = new ArrayList<>();
    try(ClientSession session =
BaseXSessionUtil.getSession()){
        String resultado = session.query(query).execute();
        //si no hay resultados, se devuelve la lista vacia
        if (resultado == null ||
resultado.trim().isEmpty()) {
            return lista;
        }
        //envolvemos en un envoltorio el resultado porque
contendrá varios nodos y daría problemas al ser un xml mal
formado
        String envoltorioXML =
"<root>" + resultado + "</root>";

        DocumentBuilderFactory factory =
DocumentBuilderFactory.newInstance();
        DocumentBuilder builder =
factory.newDocumentBuilder();
        Document doc = builder.parse(new
ByteArrayInputStream(envoltorioXML.getBytes(StandardCharsets.U
TF_8)));

        //obtenemos todos los nodos de la entidad
        NodeList nodeList =
doc.getElementsByTagName(getEntityTag());
        for (int i = 0; i < nodeList.getLength(); i++) {
            Element element = (Element) nodeList.item(i);
            //convertimos el nodo a String usando un
Transformer
            TransformerFactory tf =
TransformerFactory.newInstance();
            Transformer transformer = tf.newTransformer();

```



```

transformer.setOutputProperty(OutputKeys.OMIT_XML_DECLARATION,
"yes");

        StringWriter writer = new StringWriter();
        //transformamos el elemento xml en un string
        con el StringWriter
        transformer.transform(new DOMSource(element),
new StreamResult(writer));

        String xmlString =
writer.getBuffer().toString();

        T entity = this.getMapper().deXML(xmlString);
        lista.add(entity);
    }
} catch (Exception e) {
    e.printStackTrace();
}
return lista;
}

/**
 * Funcion que obtiene la ruta interna donde se encuentra
una entidad en la Base de datos
 * @param t La entidad
 * @return La ruta donde la entidad se encuentra
 */
public String obtenerRutaEntidad(T t) {
    String url = null;

        try(ClientSession session =
BaseXSessionUtil.getSession()) {
            url = session.query("for $doc in
collection('"+BaseXSessionUtil.DATABASE+"')//"+this.getEntityT
ag()+" where $doc/id = '"+t.getId()+" and
starts-with(db:path($doc), '"+this.getCollectionPath()+"')
return db:path($doc)").execute();
        } catch (Exception e) {
            e.printStackTrace();
        }
        return url;
    }
}

```

Y luego, las entidades herederas tienen que heredar de la clase abstracta que es el repositorio genérico para implementar sus métodos y definir cuál será la ruta de las entidades hijas en la base de datos.

```
package com.spachecor.ejerciciofinalsgb.model.dao;

import com.spachecor.ejerciciofinalsgb.model.entity.Libro;
import
com.spachecor.ejerciciofinalsgb.model.mapper.LibroMapper;
import com.spachecor.ejerciciofinalsgb.model.mapper.Mapper;
import
com.spachecor.ejerciciofinalsgb.model.repository.BaseXSessionU
til;

import java.util.List;

/**
 * Clase LibroDAOGenericImpl que se encarga de definir el
 * Mapper, el CollectionPath y el EntityTag relacionados con el
 * objeto Entidad tipo Libro para acceder a los libros.
 * @author Selene
 * @version 1.0
 */
public class LibroDAOGenericImpl extends
EntidadGenericDAOImpl<Libro> {

    private final Mapper<Libro> MAPPER_LIBRO;

    public LibroDAOGenericImpl() {
        this.MAPPER_LIBRO = new LibroMapper();
    }

    @Override
    protected Mapper<Libro> getMapper() {
        return this.MAPPER_LIBRO;
    }

    @Override
    protected String getCollectionPath() {
        return "librarywithcollections/books";
    }

    @Override
    protected String getEntityTag() {
```

```

        return "book";
    }

    public List<Libro> obtenerLibrosPorCategoria(String
categoria) {
        String xquery =
"collection('"+BaseXSessionUtil.DATABASE+"'//"+this.getEntity
Tag()+"[starts-with(db:path(.),
'"+this.getCollectionPath()+"/"+categoria+"')]";
        System.out.println(xquery);
        return super.getListAPartirDeQuery(xquery);
    }
}

```

Por otro lado, se ha creado una interfaz Mapper, que define los métodos para mapear una entidad a XML y un XML a objeto tipo entidad, todo esto genérico.

```

package com.spachecor.ejerciciofinalsgb.model.mapper;

import com.spachecor.ejerciciofinalsgb.model.entity.Entidad;
import org.w3c.dom.Document;
import org.w3c.dom.Element;

import javax.xml.parsers.DocumentBuilder;
import javax.xml.parsers.DocumentBuilderFactory;
import java.io.ByteArrayInputStream;
import java.nio.charset.StandardCharsets;

/**
 * Interfaz Mapper que define las funciones necesarias para el
 mapeo de una Entidad de tipo T
 * @param <T> El tipo de Entidad
 * @author Selene
 * @version 1.0
 */
public interface Mapper<T extends Entidad> {
    /**
     * Funcion que convierte una cadena que representa un
 objeto en formato XML a una Entidad del tipo T
     * @param xml La cadena que representa el objeto Entidad en
 XML
     * @return El objeto Entidad del tipo T
     */
    T deXML(String xml);
}

```

```

    /**
     * Funcion que convierte un objeto Entidad del tipo T en
    una cadena que representa el objeto en formato XML
     * @param t El objeto Entidad de tipo T
     * @return La cadena que representa el objeto en XML
     */
    String aXML(T t);

    /**
     * Funcion que obtiene el elemento raiz de un documento xml
     * @param xml El documento xml en formato String
     * @return El elemento raiz
     * @throws Exception La excepcion que puede lanzar
     */
    default Element obtenerElementoRaiz(String xml) throws
Exception {
        DocumentBuilderFactory factory =
DocumentBuilderFactory.newInstance();
        DocumentBuilder builder = factory.newDocumentBuilder();
        Document doc = builder.parse(new
ByteArrayInputStream(xml.getBytes(StandardCharsets.UTF_8)));
        return doc.getDocumentElement();
    }
}

```

```

package com.spachecor.ejerciciofinalsgb.model.mapper;

import com.spachecor.ejerciciofinalsgb.model.entity.Libro;
import org.w3c.dom.Element;

/**
 * Clase LibroMapper que sirve para definir como se mapea un
libro de XML a Libro y viceversa.
 * @author Selene
 * @version 1.0
 */
public class LibroMapper implements Mapper<Libro> {
    @Override
    public Libro deXML(String xml) {
        Libro libro = new Libro();
        try {
            Element root = this.obtenerElementoRaiz(xml);

```

```

libro.setId(Integer.parseInt(root.getElementsByTagName("id").item(0).getTextContent()));

libro.setTitulo(root.getElementsByTagName("title").item(0).getTextContent());

libro.setAutor(root.getElementsByTagName("author").item(0).getTextContent());

libro.setAnioPublicacion(Integer.parseInt(root.getElementsByTagName("publicationYear").item(0).getTextContent()));

libro.setCategoria(root.getElementsByTagName("category").item(0).getTextContent());

libro.setSubCategoria(root.getElementsByTagName("subcategory").item(0).getTextContent());

libro.setIsbn(Long.parseLong(root.getElementsByTagName("isbn").item(0).getTextContent()));

libro.setEditorial(root.getElementsByTagName("publisher").item(0).getTextContent());

libro.setNumeroPaginas(Integer.parseInt(root.getElementsByTagName("pages").item(0).getTextContent()));

libro.setNumeroCopiasDisponibles(Integer.parseInt(root.getElementsByTagName("availableCopies").item(0).getTextContent()));
    } catch (Exception e) {
        e.printStackTrace();
    }
    return libro;
}

@Override
public String aXML(Libro libro) {
    return "<book>" +
        "<id>" + libro.getId() + "</id>" +
        "<title>" + libro.getTitulo() + "</title>" +
        "<author>" + libro.getAutor() + "</author>" +
        "<publicationYear>" +
libro.getAnioPublicacion() + "</publicationYear>" +

```

```

        "<category>" + libro.getCategoria() +
"</category>" +
        "<subcategory>" + libro.getSubCategoria() +
"</subcategory>" +
        "<isbn>" + libro.getIsbn() + "</isbn>" +
        "<publisher>" + libro.getEditorial() +
"</publisher>" +
        "<pages>" + libro.getNumeroPaginas() +
"</pages>" +
        "<availableCopies>" +
libro.getNumeroCopiasDisponibles() + "</availableCopies>" +
        "</book>";
    }
}

```

Con esta estructura conseguimos realizar consultas a la base de datos usando objetos del tipo entidad, y poder pasar fácilmente de objeto a XML, y viceversa.

4- Gestión de colecciones

Para gestionar las colecciones, se ha creado una clase ColeccionesManager, que se encarga de gestionar el CRUD de las colecciones. Realmente las colecciones como tal en BaseX no existen, se han simulado con rutas internas. Entonces esta clase gestiona la abstracción que se ha realizado para simular las colecciones en BaseX. En nuestro Gestor de Bibliotecas, tenemos una colección principal que es nuestra base de datos. Luego, tenemos tres colecciones hijas, que son los Libros, los Usuarios y los Préstamos. Por último, colgando de Libro, tenemos todas las subcolecciones que queramos, que coinciden con las categorías de los libros.

```

package com.spachecor.ejerciciofinalsgb.model.collections;

import
com.spachecor.ejerciciofinalsgb.model.dao.LibroDAOGenericImpl;
import
com.spachecor.ejerciciofinalsgb.model.documents.DocumentosManager;
import com.spachecor.ejerciciofinalsgb.model.entity.Libro;
import
com.spachecor.ejerciciofinalsgb.model.repository.BaseXSessionUtil;
import org.basex.api.client.ClientSession;

/**

```

```

* Clase ColeccionesManager, que se encarga de gestionar el
CRUD de las colecciones(simuladas, porque basex no tiene
* colecciones como tal) en la base de datos.
* @author Selene
* @version 1.0
*/
public class ColeccionesManager {
    /**
     * Funcion que obtiene una lista con todas las colecciones
    principales, es decir, Libro, Usuario y Préstamo
     * @return La lista con las colecciones principales
     */
    public static String[] listarColecciones(){
        try(ClientSession session =
BaseXSessionUtil.getSession()){
            return session.query("distinct-values(for $doc in
collection('"+BaseXSessionUtil.DATABASE+"' ) " +
                "let $parts := tokenize(db:path($doc), '/')
" +
                                "return
concat($parts[2]))").execute().split("\\r?\\n");
        }catch (Exception e){
            e.printStackTrace();
            return null;
        }
    }

    /**
     * Funcion que se encarga de listar las subcolecciones, es
    decir, las colecciones dentro de la coleccion de libros,
     * que representan las categorias de los libros
     * @return La lista con las categorias de libros, que son
    las subcolecciones de libros
     */
    public static String[] listarSubColeccionesLibros(){
        try(ClientSession session =
BaseXSessionUtil.getSession()){
            return session.query("distinct-values(for $doc in
collection('"+BaseXSessionUtil.DATABASE+"')//book[starts-with(
db:path(.), '"+BaseXSessionUtil.DATABASE+"/books')] " +
                "let $parts := tokenize(db:path($doc), '/')
" +
                                "return
concat($parts[3]))").execute().split("\\r?\\n");

```

```

        } catch (Exception e) {
            e.printStackTrace();
            return null;
        }
    }

    /**
     * Funcion que modifica el nombre de una
     subcoleccion(realmente modifica la ruta del documento)
     * @param url La ruta interna donde se encuentra la
     subcoleccion
     * @param nuevaSubColeccion El nuevo nombre de la coleccion
     * @param viejaSubColeccion El antiguo nombre de la
     coleccion
     */
    private static void modificarSubColeccion(String url,
String nuevaSubColeccion, String viejaSubColeccion){
        DocumentosManager.modificarDocumento(url,
url.replace(viejaSubColeccion, nuevaSubColeccion));
    }

    /**
     * Funcion que se encarga de modificar la categoria(o
     subcoleccion) una entidad de tipo Libro, que realmente
     modifica
     * su ruta interna en la base de datos
     * @param libro La entidad de tipo Libro
     * @param nuevaSubColeccion El nuevo nombre de la categoria
     o subcoleccion
     * @param viejaSubColeccion El antiguo nombre de la
     categoria o subcoleccion
     */
    public static void modificarSubColeccionPorId(Libro libro,
String nuevaSubColeccion, String viejaSubColeccion){
        LibroDAOGenericImpl libroDao = new
LibroDAOGenericImpl();
        String url = libroDao.obtenerRutaEntidad(libro);
        ColeccionesManager.modificarSubColeccion(url,
nuevaSubColeccion, viejaSubColeccion);
    }

    /**

```



```

        * Funcion que se encarga de modificar la categoria o
        subcoleccion de todos los miembros de una subcoleccion
        completa
        * @param nuevaSubColeccion El nuevo nombre de la
        subcoleccion
        * @param viejaSubColeccion El antiguo nombre de la
        subcoleccion
        */

        public static void
        modificarTodosLosMiembrosSubcoleccion(String
        nuevaSubColeccion, String viejaSubColeccion){
            String[] rutas = null;
            try(ClientSession session =
            BaseXSessionUtil.getSession()){
                rutas = session.query(
                    "for $doc in
                    collection('"+BaseXSessionUtil.DATABASE+"' ) " +
                    "where starts-with(db:path($doc),
                    '"+BaseXSessionUtil.DATABASE+"/books/"+viejaSubColeccion+"' ) "
                    +
                    "return
                    db:path($doc)").execute().split("\\r?\\n");
            } catch (Exception e){
                e.printStackTrace();
            }
            if(rutas != null){
                for(String ruta : rutas){
                    ColeccionesManager.modificarSubColeccion(ruta,
                    nuevaSubColeccion, viejaSubColeccion);
                }
            }
        }

        /**
        * Funcion que elimina una subcoleccion completa, con todas
        las entidades albergadas en ella
        * @param subColeccion El nombre de la subcoleccion a
        eliminar
        */

        public static void eliminarSubcoleccion(String
        subColeccion){
            try(ClientSession session =
            BaseXSessionUtil.getSession()){
                String[] rutas = session.query(

```

```

                                "for $doc in
collection('"+BaseXSessionUtil.DATABASE+"' ) " +
                                "where starts-with(db:path($doc),
 '"+BaseXSessionUtil.DATABASE+"/books/"+subColeccion+"' ) " +
                                "return
db:path($doc)").execute().split("\\r?\\n");
        for(String ruta : rutas){
            DocumentosManager.eliminarDocumento(ruta);
        }
    } catch (Exception e){
        e.printStackTrace();
    }
}
}

```

5- Gestión de documentos XML

La gestión de los documentos XML en nuestra base de datos BaseX se realiza con la clase DocumentosManger, que contiene un CRUD para los documentos. Es importante no confundir los documentos con las colecciones en esta base de datos, porque, aunque tienen mucho que ver, no son lo mismo. Una colección puede contener uno o más documentos (nunca 0, si no no existe porque las colecciones se han simulado con rutas, y si no hay al menos un archivo, no puede existir la colección), mientras que los documentos son únicos, y varios pueden pertenecer a una misma colección.

```

package com.spachecor.ejerciciofinalsgb.model.documents;

import com.spachecor.ejerciciofinalsgb.model.entity.Entidad;
import com.spachecor.ejerciciofinalsgb.model.entity.Libro;
import com.spachecor.ejerciciofinalsgb.model.entity.Prestamo;
import com.spachecor.ejerciciofinalsgb.model.entity.Usuario;
import
com.spachecor.ejerciciofinalsgb.model.repository.BaseXSessionU
til;
import org.basex.api.client.ClientSession;

/**
 * Clase DocumentosManager, que se encarga de gestionar el CRUD
de los documentos en nuestra base de datos BaseX
 * @author Selene
 * @version 1.0
 */
public class DocumentosManager {

```

```

/**
 * Funcion que lista todos los documentos contenidos en
nuestra base de datos
 * @return Un array de Strings que contiene la lista de
documentos albergados en nuestra base de datos
 */
public static String[] listarDocumentos(){
    try(ClientSession session =
BaseXSessionUtil.getSession()){
        return session.query("for $doc in
collection('"+BaseXSessionUtil.DATABASE+"' ) " +
                                "return
db:path($doc)").execute().split("\\r?\\n");
    } catch (Exception e){
        e.printStackTrace();
        return null;
    }
}

/**
 * Funcion que se encarga de agregar un nuevo documento en
la base de datos
 * @param clase La clase del objeto de tipo Entidad al que
pertenecen las entidades que se van a agregar a la base
de datos
 * @param url La url donde se encuentra el documento
 * @param subCategoria La subcategoria a la que pertenece
la entidad, que puede ser null si no pertenece a ninguna,
como pasaría si introducimos libros
o préstamos
 * @param <T> Objetos que hereden de Entidad
 */
public static <T extends Entidad> void
agregarDocumento(Class<T> clase, String url, String
subCategoria){
    try(ClientSession session =
BaseXSessionUtil.getSession()){
        String rutaInterna = BaseXSessionUtil.DATABASE+"/";
        if(clase.equals(Libro.class))
rutaInterna+="books/" + ((subCategoria==null)?"":subCategoria+"/
");
        else if(clase.equals(Usuario.class))
rutaInterna+="users/";
    }
}

```

```

        else if(clase.equals(Prestamo.class))
rutaInterna+="loans/";
        session.execute("add to "+rutaInterna+" "+url);
    } catch (Exception e) {
        e.printStackTrace();
    }
}

/**
 * Funcion que define como se modifica la ruta de un
documento en nuestra base de datos
 * @param antiguaRuta La ruta interna anterior
 * @param nuevaRuta La nueva interna ruta
 */
public static void modificarDocumento(String antiguaRuta,
String nuevaRuta) {
        try(ClientSession session =
BaseXSessionUtil.getSession()) {
            session.execute("rename "+antiguaRuta+"
"+nuevaRuta);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }

/**
 * Funcion que define como se elimina un documento de la
base de datos
 * @param ruta La ruta interna donde se encuentra la base
de datos
 */
public static void eliminarDocumento(String ruta) {
        try(ClientSession session =
BaseXSessionUtil.getSession()) {
            session.execute("delete "+ruta);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }

/**
 * Funcion que exporta la base de datos y su estructura
 * @param url La URL donde almacenar la base de datos
 */
public static void exportarBBDD(String url) {

```

```

                                try(ClientSession session =
BaseXSessionUtil.getSession()) {
    System.out.println("export \""+url+"\"");
    session.execute("export \""+url+"\"");
} catch (Exception e) {
    e.printStackTrace();
}
}
}

```

6- Conclusión

El desarrollo de un sistema de gestión de bibliotecas basado en una base de datos XML con BaseX ha permitido establecer una estructura flexible y eficiente para el almacenamiento y consulta de información. A través de la implementación de clases específicas para la conexión, gestión de documentos y organización de colecciones, se ha logrado un sistema modular y escalable.

La integración de un CRUD genérico facilita la manipulación de entidades dentro de la base de datos, garantizando un acceso ágil y seguro a los registros de libros, usuarios y préstamos. Además, la simulación de colecciones en BaseX ha demostrado ser una solución efectiva para organizar los datos sin comprometer el rendimiento del sistema.

En el futuro, se podrían implementar mejoras como la posibilidad de exportar toda la base de datos a un único documento XML en lugar de un árbol de carpetas y archivos individuales. También se podrían añadir más filtros avanzados para la búsqueda de documentos, facilitando una recuperación de información más precisa y eficiente. Estas mejoras permitirían optimizar aún más el rendimiento y la usabilidad del sistema.